

BOOK STORE SOLUTION

This document contains the technical details about solution that integrates React 18 + Redux and .NET 10 Web API using PostgreSQL and Redis cache system to optimise performance focused on large applications.

Contents

Requirements.....	2
Overview.....	3
Scope.....	4
RQ-01 Web API.....	5
RQ-02 React Web App.....	7
UI Screens.....	8
RQ-03 Redis.....	14
RQ-04 PostgreSQL.....	15
RQ-05 JWT Authentication.....	16
RQ-06 Docker.....	17
RQ-07 ALC.....	18
Annexes.....	19
Watchtower.....	19

Doc. Version	Date	Author	Description
2.0	2025-02-03	https://github.com/kraftcoding	Second version of the release

Requirements

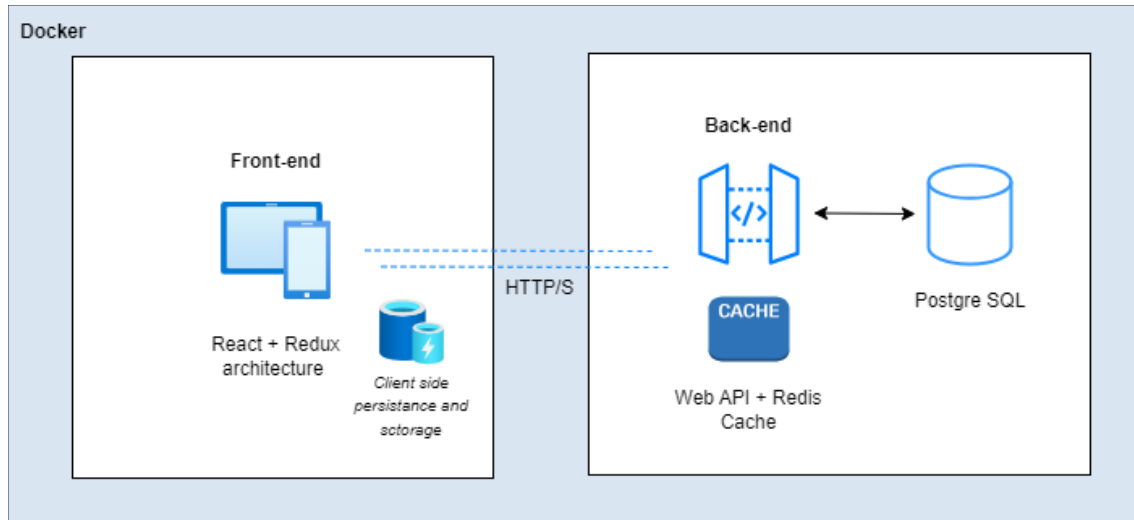
The next table shows the requirements for the solution described.

Requirement	Description
RQ-001 Web API	Secured endpoints to authenticate and manage applications data
RQ-002 React-Redux App	React based front-end integrating redux for data storage and persistence
RQ-003 Redis	Server side in-memory storage system
RQ-004 PostgreSQL	Data base
RQ-005 JWT Authentication	Token bearer based authentication
RQ-006 Docker	Containerisation and deployment
RQ-006 ALC	Application life-cycle definition

Overview

This document contains the technical details of the Book Store 2.0 full-stack solution, comprised of .Net 10 based Web API back-end, React 18 and Redux architecture based front-end application, PostgreSQL as database server, Redis for caching and in-memory data exchange to authorise and access to information and docker to deployment.

The next image the different elements the solutions comprised of and how they are distributed in the overall diagram.



As we can see, the solution offers scalability and performance features by using Redux for client side persistence and storage together with server side cache with Redis. This is a key point to take in account for optimising performance in both sides especially when it comes to larger applications with big amount of data and fast response requirements.

User can access the application with the React based UI, being this fully responsive for diverse devices. The data requests goes through HTTP channels to server side using the cache to minimise database access.

The main features of the solution are:

- Built with .NET 10 and React 18: Utilizes the latest features for efficient development.
- Redux Toolkit: Manages application state effectively.
- Docker Compose: Manages multi-container applications seamlessly.
- PostgreSQL: Powerful relational database for data storage.
- Redis: Caching solution for improved performance.
- JWT Authentication and Authorization: Secure access to API endpoints.
- CI/CD with GitHub Actions: Automated workflows for building, testing, and deploying the application.
- Entity Framework Core: Simplifies data access with ORM capabilities.

- **Swagger Integration:** Interactive API documentation for easy testing and exploration.
- **Responsive React Frontend:** User-friendly interface for managing books.
- **Error Handling and Logging:** Robust mechanisms for tracking and resolving issues.
- **Scalability:** Designed to handle increased load with ease.
- **Modular Codebase:** Organized structure for maintainability and extensibility.

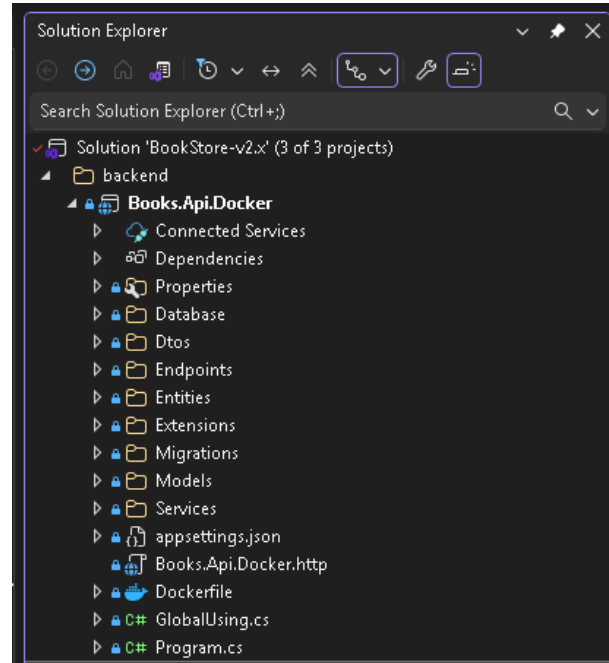
Scope

The aim of this development is to offer a highly scalable and performance focused architecture as solution template for generic web applications to manage and store data by a responsive UI. Specifying application life-cycle best practices for containerised deployments with Docker.

RQ-01 | Web API

The Web API is built with .NET 10 for a faster runtime and automatic memory pool eviction to reduce memory footprint in long-running applications minimising Garbage Collection (GC) overhead.

The project structure is the next one.



As we can see, it is designed by using clean architecture principles. The domain and application layer as the center of the design are the core of the project, being independent of the data access and infrastructure concerns. The Docker file contains the build and containerization instructions to assemble the image.

In the app-settings we will find the connection strings for local and dockerised environments, and JWT configuration. It uses Swagger to get the API definition and schema together with a UI to test the endpoints.

Implement a cache system through Redis to optimize response times and resources by minimizing queries on the data base.

Swagger UI

localhost:5225/swagger/index.html

Swagger
Sponsored by SMARTBEAR

Select a definitionBooks.Api.Docker v1

Books.Api.Docker

1.0OAS 3.0

http://localhost:5225/swagger/v1/swagger.json

AuthEndpoints

POST/api/Auth

Parameters

No parameters

Request body required

application/json

Example Value | Schema

```
{  "name": "string",  "email": "string",  "password": "string",  "initials": "string"}
```

Responses

Code	Description	Links
200	OK	No links

POST/api/Auth/loginUser

PUT/api/Auth/updateProfile

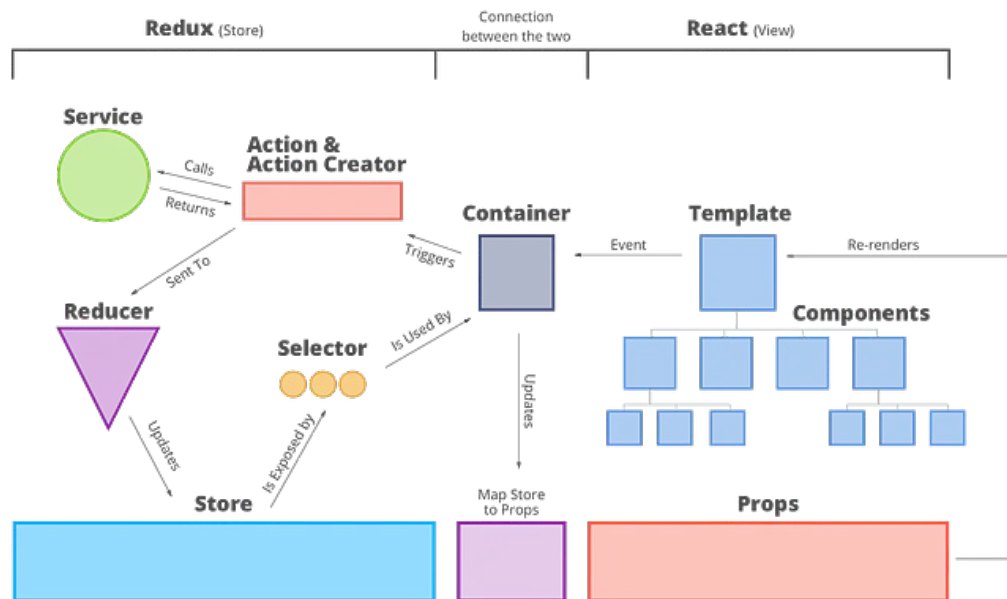
GET/api/Auth/version

RQ-02 | React Web App

The front-end is built with React-Redux for a better performance and scalability.

Developing with React, is more about the architecture than the actual view being comprised of other parts that wire things up to that view. There are a lot of parts there, but each plays a very specific purpose.

Next diagram shows the parts by which the applications is comprised of and their relationship.



As we can see Services, Actions & Action Creators, Reducers, and Selectors are Redux. Templates and Components are react. The Container is piece that allows them to be loosely coupled together. It allows the store and the view to be able to change and grow independently of each other.

In this kind of architecture the state is in the store for Redux, and in the props for React. When the store is updated it re-maps through the container and re-renders the DOM. When there is an event in the DOM that triggers events through the container and to the Actions. The biggest thing to know here is that the store is where redux stores the data, the data is mapped to props, where React displays through components. And all that is state.

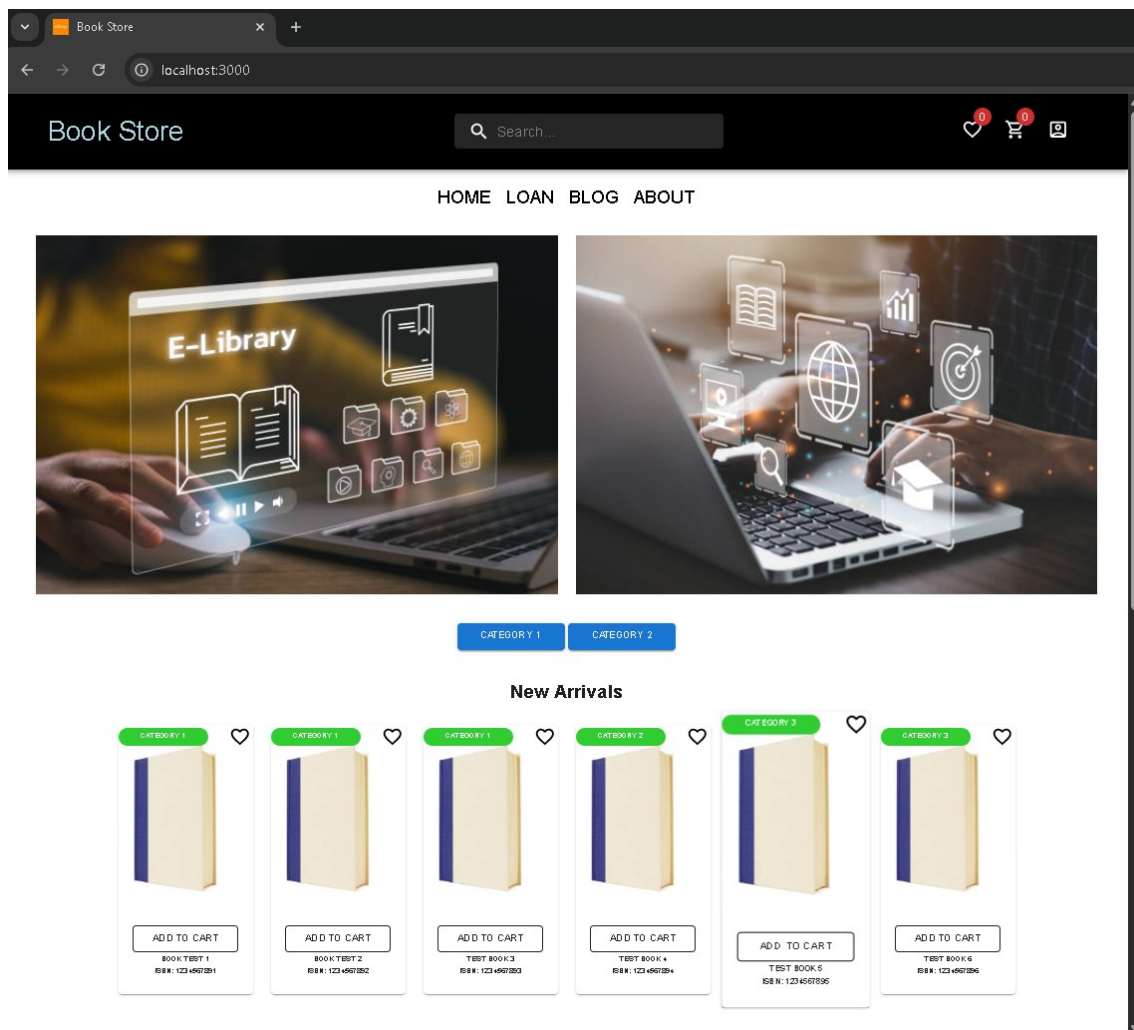
The correspondent part and connections are the next ones.

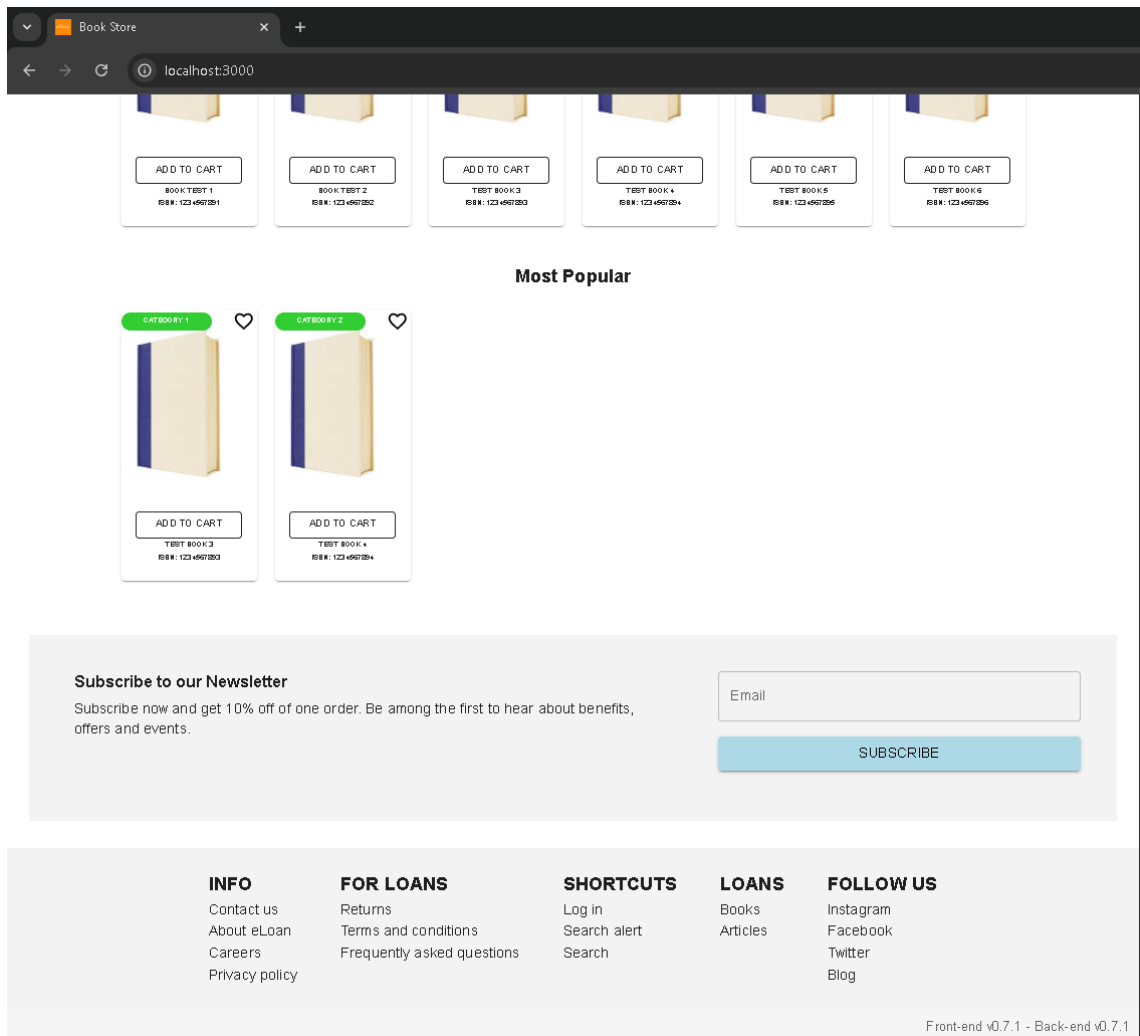
- **Component:** The component is rendered with a set of props that get passed down from it's parent component.
- **Template:** Special kind of component that does things more specifically than the lower components do.

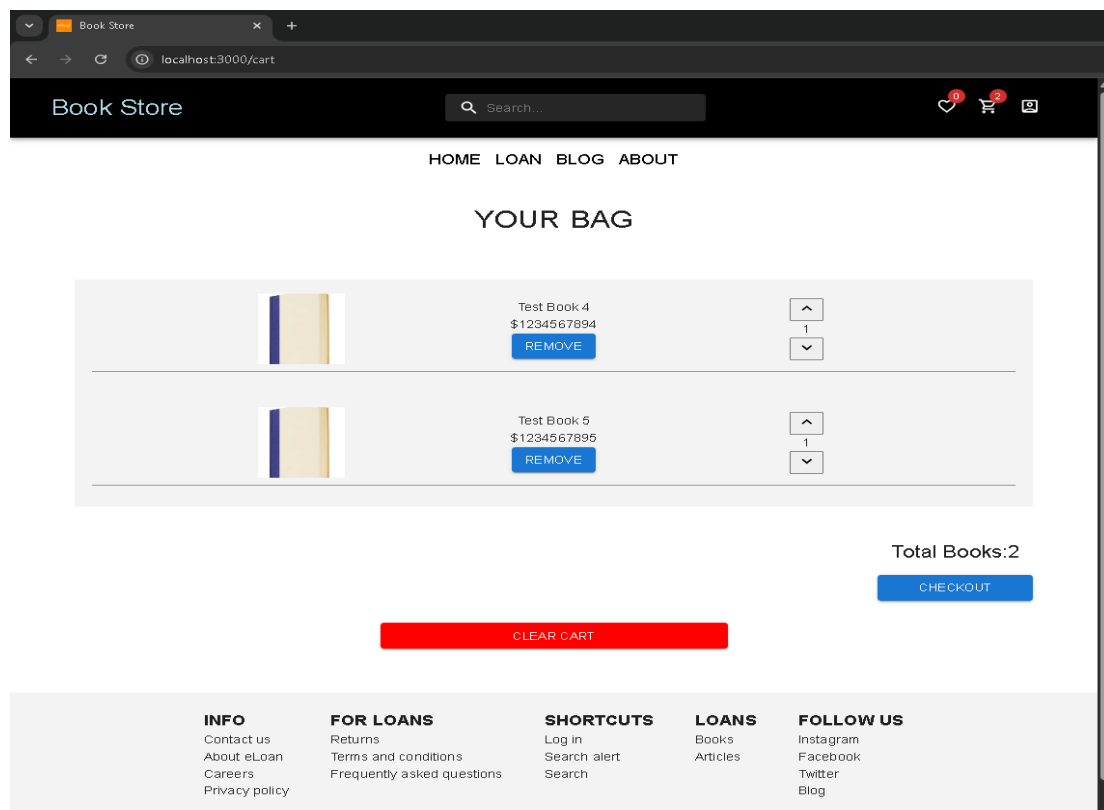
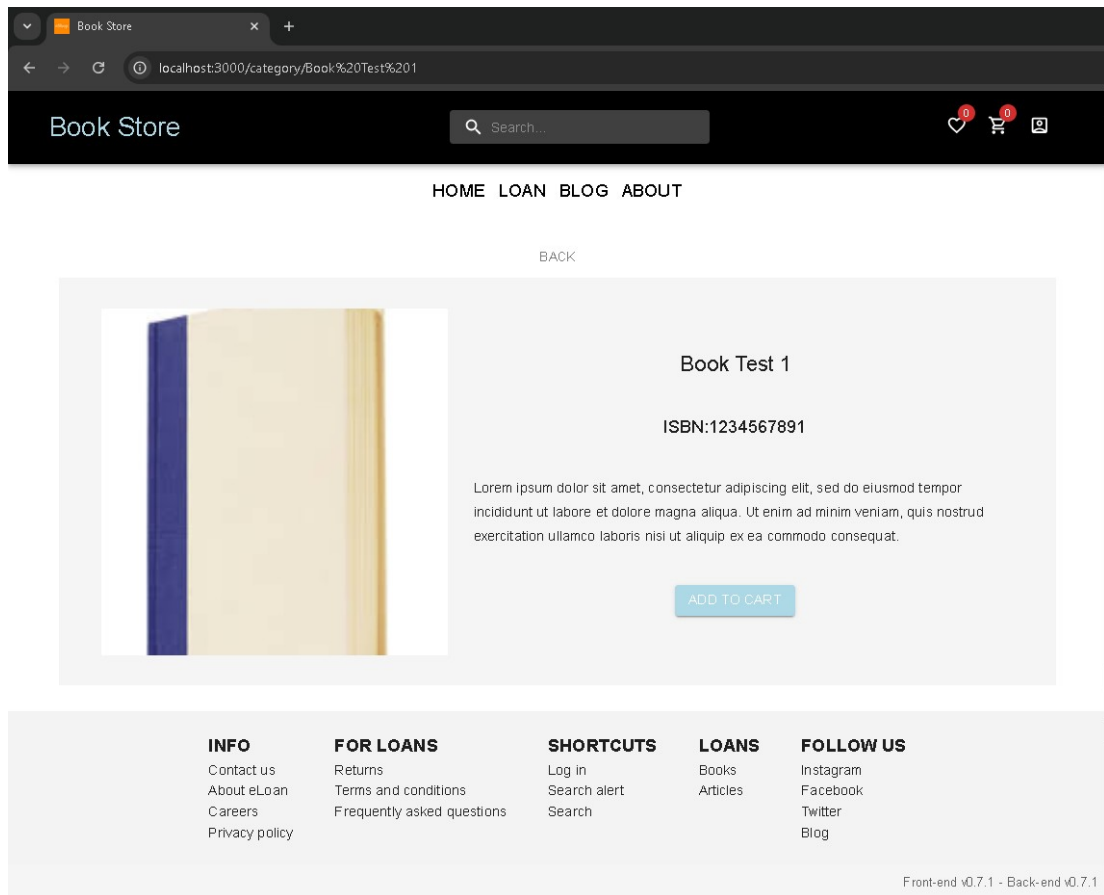
- **Container:** The container is the glue that connects react to redux. It connects it in a lot of ways.
- **Actions:** This is an object that contains the type of action and the state that was changed because of the action.
- **Reducers:** For every dispatch every reducer is called and given the dispatched action. Then it is up to the reducer to handle it or pass it on.
- **Selector:** The selector is how you would get data out of your store in the container.

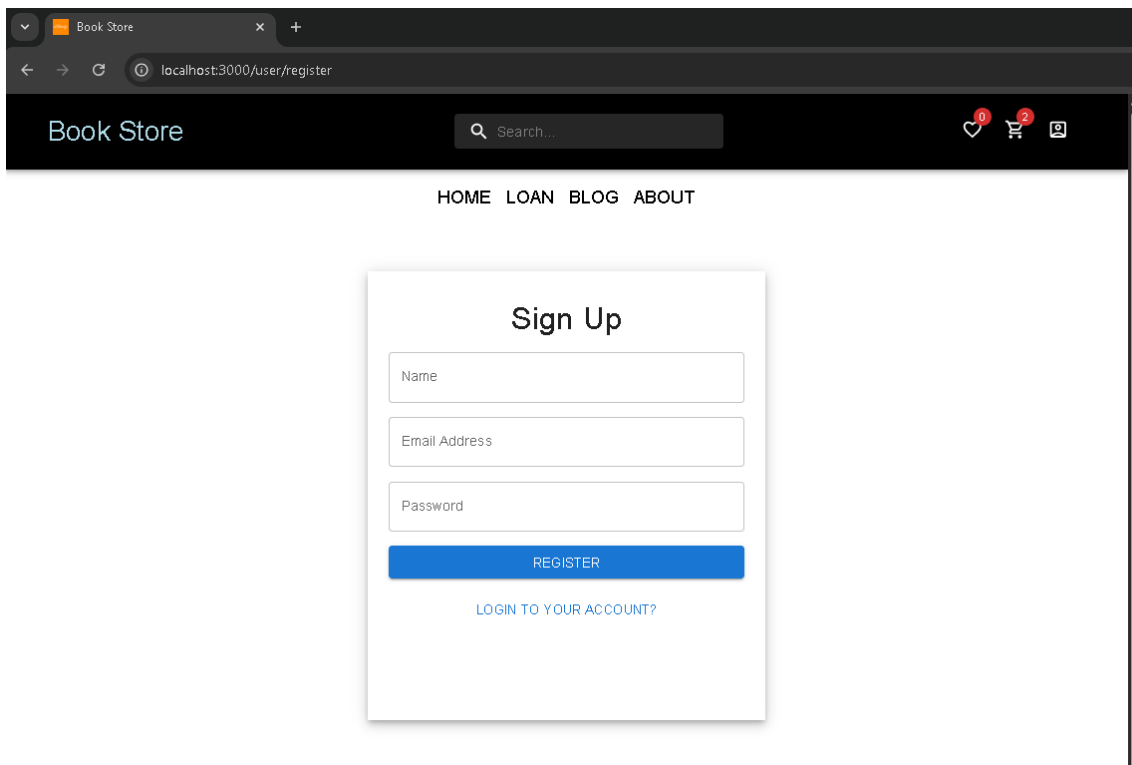
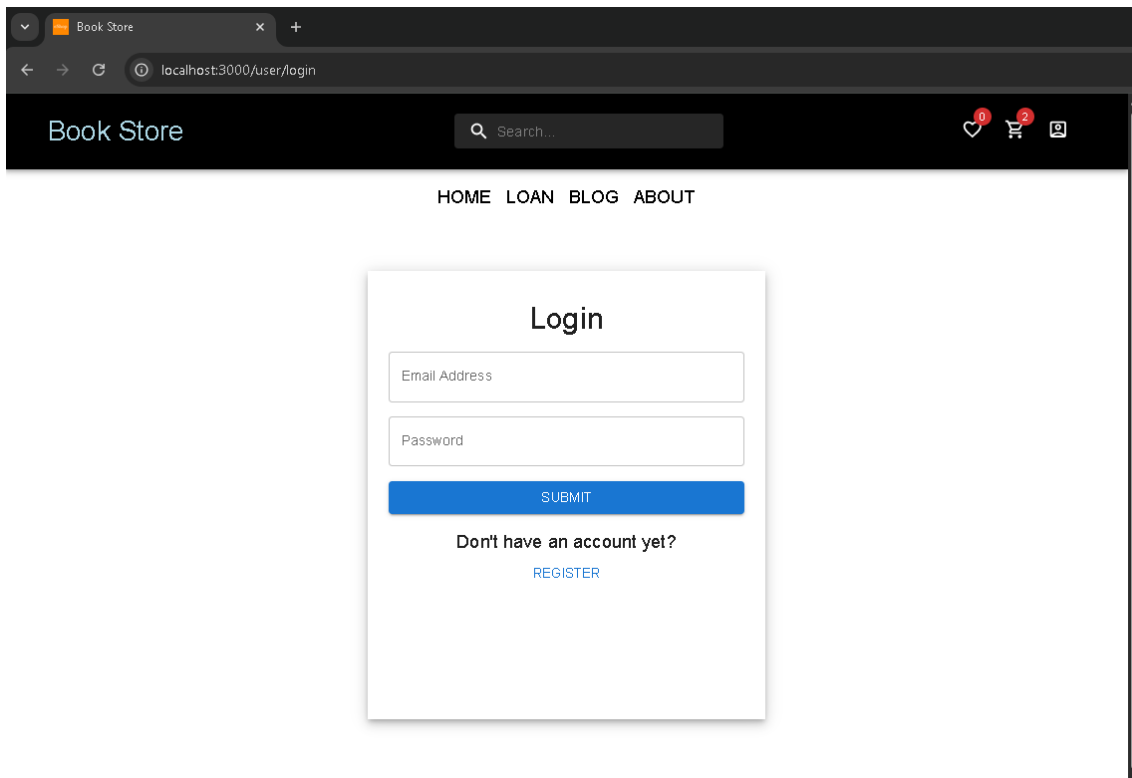
UI Screens

Next we are going to have a look to the different screens of the application.









Book Store

localhost:3000/content/book-list

Book Store

Search...

0

2

HOME LOAN BLOG ABOUT

Manage Books

Title	ISBN	Description	Author	Category	ADD BOOK	
Book Test 1	1234567891	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.	Author 1	Category 1	EDIT	DELETE
Book Test 2	1234567892	Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.	Author 1	Category 1	EDIT	DELETE
Test Book 3	1234567893	Sed ut perspiciatis, unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam eaque ipsa, quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt, explicabo.	Author 3	Category 1	EDIT	DELETE
Test Book 4	1234567894	Nemo enim ipsam voluptatem, quia voluptas sit, aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos, qui ratione voluptatem sequi nesciunt,	Author 4	Category 2	EDIT	DELETE
Test Book 5	1234567895	The Lorem ipsum text is derived from sections 1.10.32 and 1.10.33 of Cicero's De finibus bonorum et malorum.[6][7] The physical source may have been the 1914 Loeb Classical Library edition of De finibus.	Author 4	Category 3	EDIT	DELETE

Rows per page: 5
1-5 of 7
< > >|

Book Store

localhost:3000/content/book/add

Book Store

Search...

0

2

HOME LOAN BLOG ABOUT

Add Book

ISBN

Title

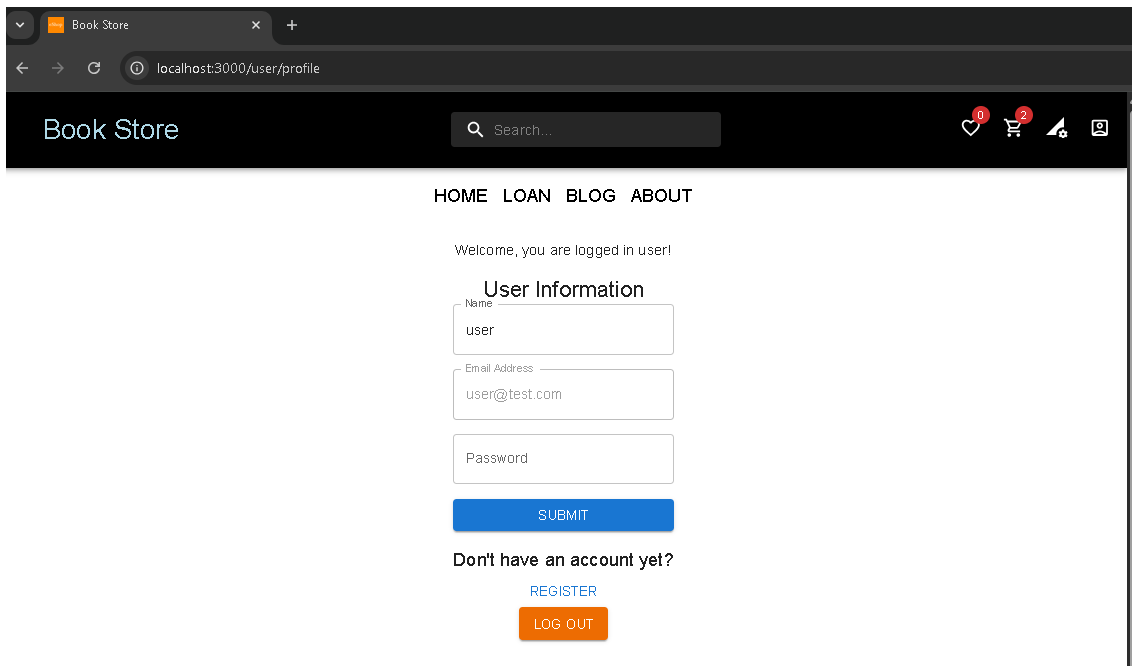
author

description

category

image

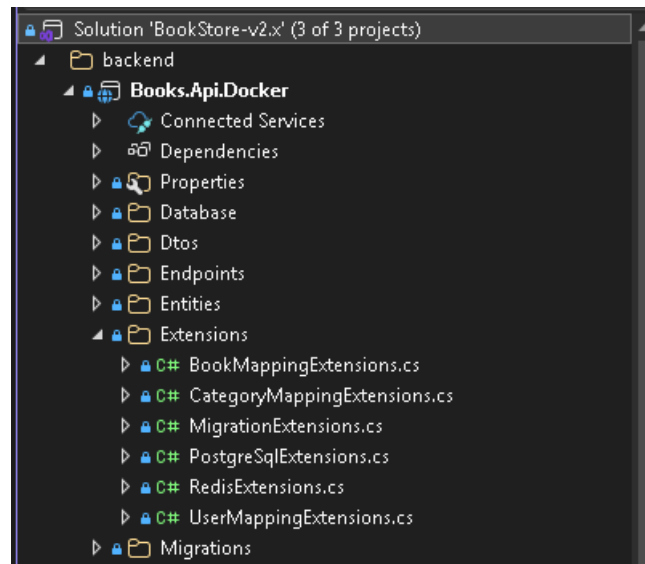
SUBMIT



RQ-03 | Redis

Redis is an open-source, in-memory NoSQL data structure store to implement a cache strategy in the system that can be included in every production-level application, and it will make the application more performant. It stores the data in key-value pairs, unlike a normal database that runs on your disk and stores all your information on disk running inside of the computer's RAM which makes it very useful for fast data exchange.

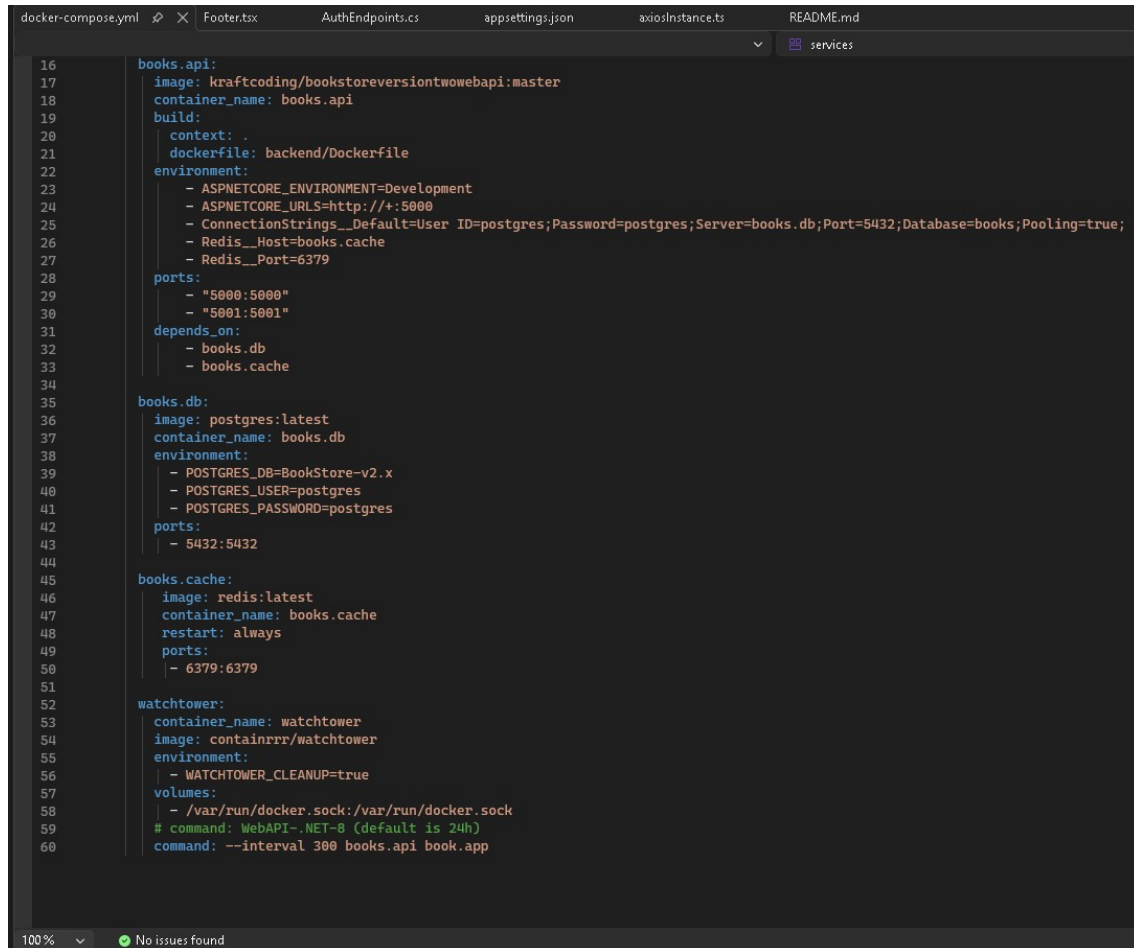
Redis is deployed by the instructions in Docker Compose and implemented as extension in the web API.



RQ-04 | PostgreSQL

PostgreSQL is a powerful, open-source object-relational database management system (ORDBMS) known for its high reliability, performance, and SQL compliance. It handles both relational (tabular) and non-relational (JSON) data, making it ideal for complex analytical, geospatial, and transactional applications.

PostgreSQL is deployed by the instructions in Docker Compose and implemented as extension in the web API.



```
16 books.api:
17   image: kraftcoding/bookstoreversiontwowebapi:master
18   container_name: books.api
19   build:
20     context: .
21     dockerfile: backend/Dockerfile
22   environment:
23     - ASPNETCORE_ENVIRONMENT=Development
24     - ASPNETCORE_URLS=http://*:5000
25     - ConnectionStrings__Default=User ID=postgres;Password=postgres;Server=books.db;Port=5432;Database=books;Pooling=true;
26     - Redis__Host=books.cache
27     - Redis__Port=6379
28   ports:
29     - "5000:5000"
30     - "5001:5001"
31   depends_on:
32     - books.db
33     - books.cache
34
35 books.db:
36   image: postgres:latest
37   container_name: books.db
38   environment:
39     - POSTGRES_DB=BookStore-v2.x
40     - POSTGRES_USER=postgres
41     - POSTGRES_PASSWORD=postgres
42   ports:
43     - 5432:5432
44
45 books.cache:
46   image: redis:latest
47   container_name: books.cache
48   restart: always
49   ports:
50     - 6379:6379
51
52 watchtower:
53   container_name: watchtower
54   image: containrrr/watchtower
55   environment:
56     - WATCHTOWER_CLEANUP=true
57   volumes:
58     - /var/run/docker.sock:/var/run/docker.sock
59   # command: WebAPI-.NET-8 (default is 24h)
60   command: --interval 300 books.api book.app
```

RQ-05 | JWT Authentication

A JSON Web Token (JWT) is an open, industry-standard (RFC 7519) method for securely representing claims between two parties as a compact, URL-safe JSON object. It is primarily used for authentication and authorisation in web applications, allowing servers to verify user identity without storing session data, making it ideal for scalable, stateless API architecture.

To implement authentication a HTTP Injector has been implemented in the Axios instance of React together with the Web API side procedures to authenticate and authorise access to content.

```
TS axiosInstance.ts x TS tokenManager.ts
src > common > TS axiosInstance.ts > [⌕] default
24 // Request interceptor - inject access token
25 apiClient.interceptors.request.use(
26   async (config: InternalAxiosRequestConfig) => {
27
28     const accessToken = tokenManager.getAccessToken();
29
30     if (accessToken) {
31       config.headers.Authorization = `Bearer ${accessToken}`;
32     }
33
34     return config;
35   },
36   (error: AxiosError) => { + (parameter) error: AxiosError<unknown, any>
37     return Promise.reject(error);
38   }
39 );
40
41 export default apiClient;
```

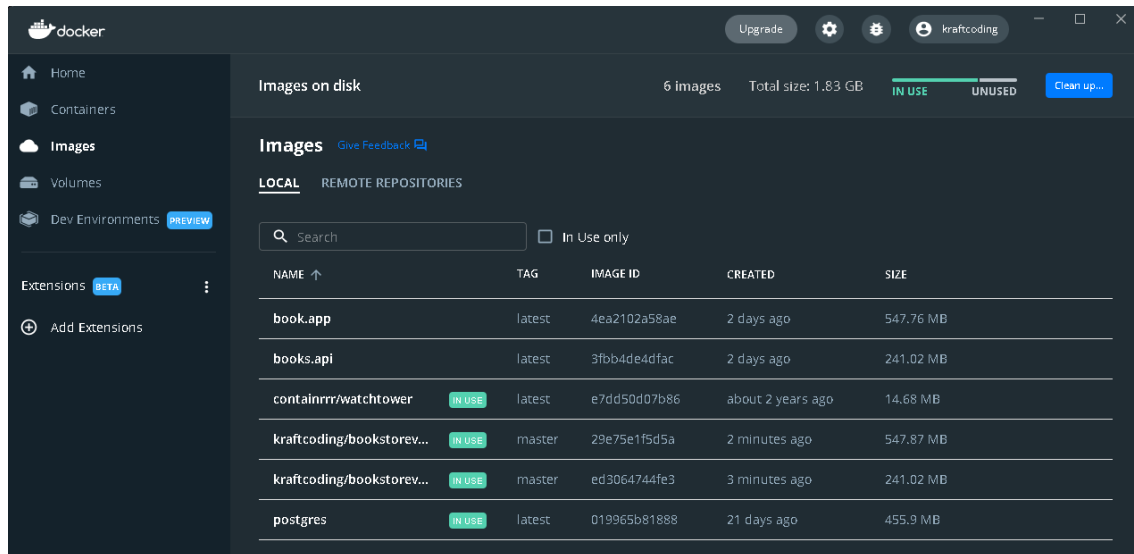
```
TS axiosInstance.ts TS tokenManager.ts x
src > common > TS tokenManager.ts > TokenManager > getAccessToken
9 class TokenManager {
23 // Get current access token
24 getAccessToken(): string | null {
25
26   if (this.isAccessTokenExpired()) {
27     return null;
28   }
29
30   // due to in classes or not function components useState procedure can't be used,
31   // this is the procedure that needs to be used in this case
32   var token = store.getState().auth?.userInfo?.token
33
34   if (token != undefined){
35     return token;
36   }
37
38   return null;
39
40 }
41
```

RQ-06 | Docker

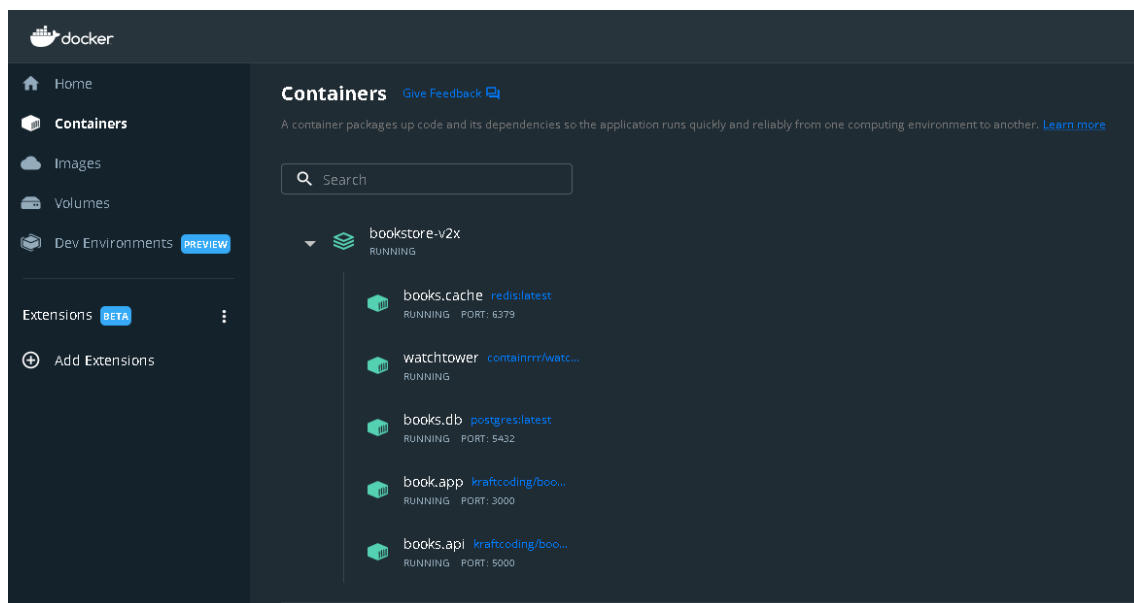
The docker images and deployment instructions are defined in the docker-compose.yml.

Docker is a software platform that allows you to build, test, and deploy applications quickly. Docker packages software into standardized units called containers that have everything the software needs to run including libraries, system tools, code, and runtime.

In this case we will use it to manage the correspondent images and deploy all the fore mentioned solution components together with Watchtower to handle image updates in the Docker Hub and update locally.

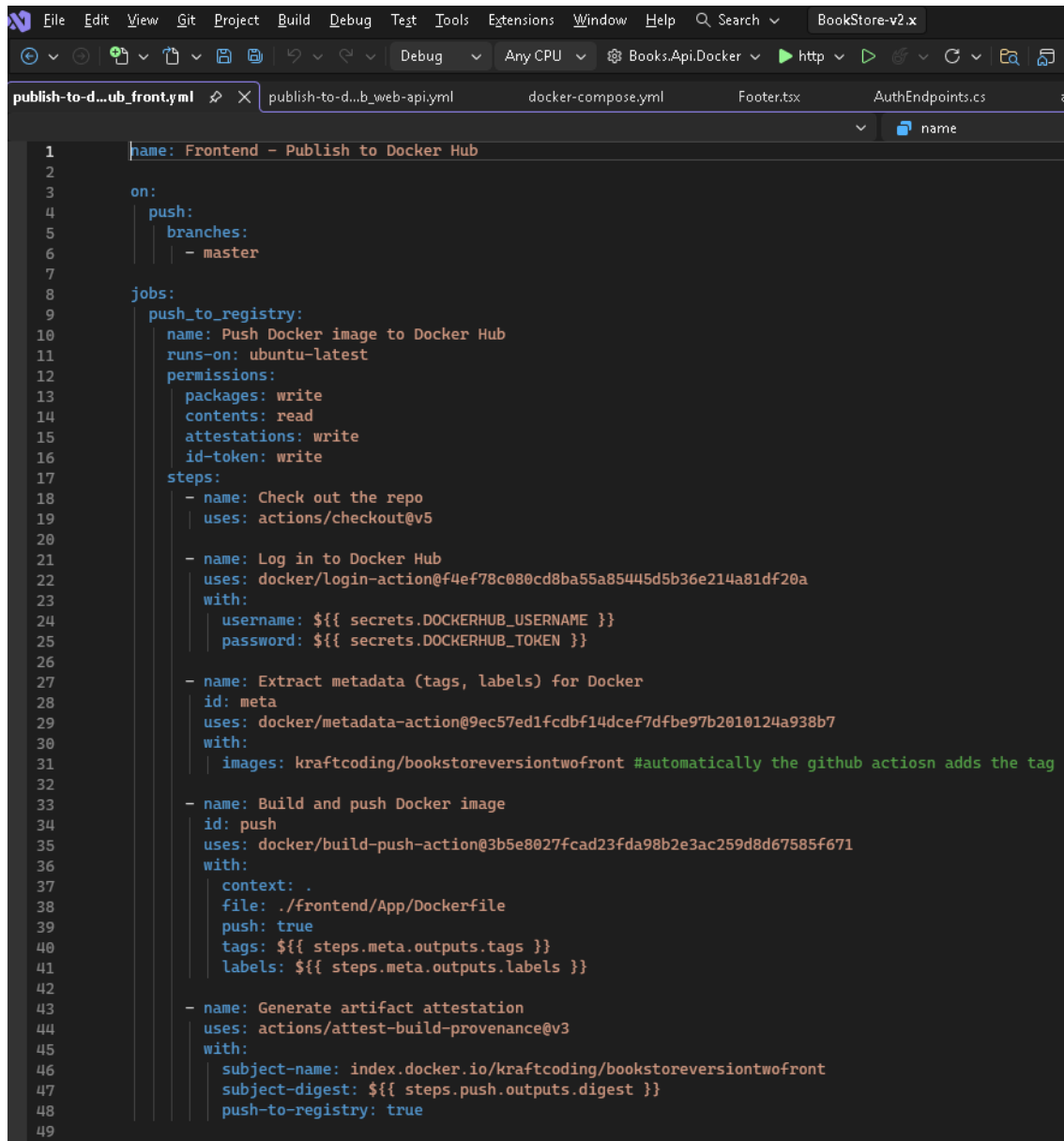


The next image shows the containers.



RQ-07 | ALC

In this case the application life-cycle is defined by the Github Actions defined in `\.github\workflows` folder and the Watchtower service which runs locally, thus, executing successfully Continuous Integration and Continuous Deployment (CI/CD) pipelines that are an essential part of streamlining software development workflows keeping the container images updated for security and performance.



```
1  name: Frontend - Publish to Docker Hub
2
3  on:
4    push:
5      branches:
6        - master
7
8  jobs:
9    push_to_registry:
10     name: Push Docker image to Docker Hub
11     runs-on: ubuntu-latest
12     permissions:
13       packages: write
14       contents: read
15       attestations: write
16       id-token: write
17     steps:
18       - name: Check out the repo
19         uses: actions/checkout@v5
20
21       - name: Log in to Docker Hub
22         uses: docker/login-action@f4ef78c080cd8ba55a85445d5b36e214a81df20a
23         with:
24           username: ${ secrets.DOCKERHUB_USERNAME }
25           password: ${ secrets.DOCKERHUB_TOKEN }
26
27       - name: Extract metadata (tags, labels) for Docker
28         id: meta
29         uses: docker/metadata-action@9ec57ed1fcd8f14dcef7dfbe97b2010124a938b7
30         with:
31           images: kraftcoding/bookstoreversiontwofront #automatically the github action adds the tag :
32
33       - name: Build and push Docker image
34         id: push
35         uses: docker/build-push-action@3b5e8027fcd23fda98b2e3ac259d8d67585f671
36         with:
37           context: .
38           file: ./frontend/App/Dockerfile
39           push: true
40           tags: ${ steps.meta.outputs.tags }
41           labels: ${ steps.meta.outputs.labels }
42
43       - name: Generate artifact attestation
44         uses: actions/attest-build-provenance@v3
45         with:
46           subject-name: index.docker.io/kraftcoding/bookstoreversiontwofront
47           subject-digest: ${ steps.push.outputs.digest }
48           push-to-registry: true
49
```

Annexes

Watchtower

Watchtower operates by monitoring the repositories of images used by your running Docker containers. When a new version of an image is pushed to the repository, Watchtower can automatically:

1. Pull the updated image
2. Gracefully stop the running container
3. Start a new container using the updated image with the same parameters
4. Remove the old container (if configured to do so)

This process ensures your containers always run the latest available versions of their base images without requiring manual intervention.

For more information:

- <https://containrrr.dev/watchtower/>